Usuarios
<<abstract>>
Usuario
-idUsuario: String
-login: String
-password: String
-nombre: String
-favoritos: List<JuegoDeMesa>
+getLogin(): String
+autenticar(loginIngresado: String, password: String): boolean
+agregarFavorito(juego: JuegoDeMesa): void
+quitarFavorito(juego: JuegoDeMesa): void
+ getFavoritos(): List<JuegoDeMesa>
+getIdUsuario(): String
+ getNombre(): String
+verificarPassword(password: String): boolean
+getPassword(): String
Cliente
-historialCompras: List<Venta>
-historialPrestamos
-puntosFidelidad: double
- idCliente: String
- bonoTorneoAmistoso: double
+agregarCompra(venta: Venta): void
+getMesaActual(): Mesa
+setMesaActual(mesa: Mesa): void
+tieneMesa(): boolean
+getPuntosFaltidad(): double
+agregarPuntos(double): void
+usarPuntos(double): boolean
+getIdCliente(): String
+ getHistorialCompras(): List<Venta>
+getBonoTorneoAmistoso(): double
+setBonoTorneoAmistoso(double bonoTorneoAmistoso):void
+ tieneBonoTorneoAmistoso(): boolean
+ limpiarBonoTorneoAmistoso(): void
<<abstract>>
Empleado
-codigoDescuento: String
-enTurno: boolean
- turno: Turno
- puntosFidelidad: double
- bonoTorneoAmistoso: double
+estaEnTurno(): boolean
+setEnTurno(valor: boolean): void
+getCodigoDescuento(): String
+getTurno(): Turno
+setTurno(turno: Turno): void
+puedePedirPrestado(): boolean
+ agregarPuntos(double puntos): void
+ usarPuntos(double puntos): boolean
+ getPuntosFidelidad(): double
+ usarBonoTorneoAmistoso(): double
+getBonoTorneoAmistoso(double bonoTorneoAmistoso):void
+ tieneBonoTorneoAmistoso(): boolean
+ limpiarBonoTorneoAmistoso(): void
Administrador
+aprobarSolicitud(solicitud: SolicitudCambioTurno): void
+rechazarSolicitud(solicitud: SolicitudCambioTurno): void
Mesero
-juegosQueConoce: List<JuegoDeMesa>
+conoceJuego(juego: JuegoDeMesa): boolean
+aprenderJuego(juego: JuegoDeMesa): void
+getJuegosQueConoce(): List<JuegoDeMesa>
Cocinero
Servicios
ServicioPrestamo
-inventarioPrestamo: InventarioPrestamo
-historialPrestamos: List<Prestamo>
-meseros: List<Mesero>
+prestarJuegoGdiPrestamo: String, usuario: Usuario, juego: JuegoDeMesa, mesa: Mesa): Prestamo
+devolverJuego(prestamo: Prestamo): void
+validarPrestamo(usuario: Usuario, juego: JuegoDeMesa, mesa: Mesa): boolean
+buscarMeseroCapacitado(juego: JuegoDeMesa, meseros: List<Mesero>): Mesero
+getHistorialPrestamos(): List<Prestamo>
+getPrestamosActivos(): List<Prestamo>
+ getMeseros(): List<Mesero>
ServicioVentas
- IVA = 0.19: double
-DESCUENTO_EMPLEADO = 0.20: double
- DESCUENTO CODIGO = 0.10: double
-inventarioVentas: InventarioVenta
-historialVentas: List<Venta>
-itemsEnProgreso: Map<String, List<ItemVenta>>
-descuentosEnProgreso: Map<String, Double>
+crearVenta(idVenta: String, comprador: Usuario, tipo: TipoVenta): Venta
+agregarItem(idVenta: String, item: ItemVenta): void
+aplicarDescuento(idVenta: String, codigoDescuento: String, comprador: Usuario): void
+finalizarVenta(idVenta: String, comprador: Usuario, tipo: TipoVenta): Venta
-calcularTotal(items: List<ItemVenta>): double
+resolverDescuento(comprador: Usuario, codigoDescuento: String): double
+getHistorialVentas(): List<Venta>
ServicioInventario
-inventarioPrestamo: InventarioPrestamo
-inventarioVenta: InventarioVenta
+ agregarCopiaAPrestamo(copia: CopiaJuego): void
+ reparar Copia(copiaAReemplazar: CopiaJuego, juego: JuegoDeMesa): boolean
+ marcarComoRobada(copia: CopiaJuego): void
+reabastecerVenta(juego: JuegoDeMesa, cantidad: int, precio: double): void
+reabastecerPrestamo(copia: CopiaJuego): void
+moverDeVentaAPrestamo(juego: JuegoDeMesa, idNuevaCopia: String): void
+actualizarEstadoCopia(copia: CopiaJuego, estado: String): void
+getCopiasPorJuego(juego: JuegoDeMesa): List<CopiaJuego>
+ getInventarioPrestamo(): InventarioPrestamo
+ getInventarioVenta(): InventarioVenta
Sugerencia
-idSugerencia: String
-empleado: Empleado
-nombrePropuesto: String
-categoria: String
-estado: EstadoSugerencia
+ EstadoSugerencia {PRENDIENTE, APROBADA, RECHAZADA}: enum
+aprobar(): void
+rechazar(): void
+getEstado(): EstadoSugerencia
+getNombrePropuesto(): String
+getCategoria(): String
+getEmpleado(): Empleado
+getIdSugerencia(): String
ServicioTorneo
- MAX_PARTICIPANTES_POR_INSCRIPCION = 3 int
- torneos: List<torneos>
- inventarioPrestamo: InventarioPrestamo
+crearTorneoAmistoso(String idTorneo, String nombre, JuegoDeMesa juego, String diaSemana, int cupoMaximo, double bonoPremio): torneos
+crearTorneoCompetitivo(String idTorneo, String nombre, JuegoDeMesa juego, String diaSemana, int cupoMaximo, double tarifaEntrada): torneos
+inscribirse(torneos torneo, Usuario usuario, int numParticipantes): boolean
+desinscribirse(torneos torneo, Usuario usuario): boolean
+registrarGanador(torneos torneo, Usuario ganador): Inscripcion
- guardarBonoPremioAmistoso(Usuario ganador, double porcentajeBono): void
+empleadoCubreTurnoEseDia(Empleado empleado, String diaTorneo): boolean
+validarCupoConCopias(JuegoDeMesa juego, int cupoMaximo): boolean
+getTorneos(): List<torneos>
+buscarTorneo(String idTorneo): torneos
+getInventarioPrestamo(): InventarioPrestamo
ServicioTurnos
- MIN_COCINEROS = 1: int
-MIN_MESEROS = 2: int
-empleados: List<Empleado>
- turnos: List<Turno>
- solicitudesPendientes: List<SolicitudCambioTurno>
+crearTurno(empleado: Empleado, dia: String): Turno
+eliminarTurno(turno: Turno): void +solicitarCambio(solicitud: SolicitudCambioTurno): void
+validarMinimosPersonal(dia: String): boolean
+aprobarSolicitud(solicitud: SolicitudCambioTurno): void
+ rechazarsolicitud(solicitud: SolicitudCambioTurno): void
+getEmpleados(): List<Empleado>
+validarMinimosPersonalSinEmpleado(empleadoSaliente: Empleado): boolean
+ getSolicitudesPendientes(): List<SolicitudCambioTurno>
+ getTurnos(): List<Turno>
ServicioCafeteria
- DESCUENTO_EMPLEADO = 0.20 : double
- DESCUENCO CODIGO = 0.10 : double
-menu: List<Alimento>
-historialVentas: List<Venta>
+validarAlimento(alimento: Alimento, mesa: Mesa): boolean
+crearPedido(idPedido: String, mesa: Mesa): Pedido
+agregarAlPedido(pedido: Pedido, alimento: Alimento, cantidad: int): void
+ finalizarPedido(idVenta: String, pedido: Pedido, comprador: Usuario, codigoDescuento: String): Venta
- resolverDescuento(Usuario comprador, String codigoDescuento): double
+agregarAlMenu(alimento: Alimento): void
+getMenu(): List<Alimento>
+getHistorialVentas(): List<Venta>
ServicioReportes
-ventasJuegos: List<Venta>
-ventasCafeteria: List<Venta>
+generarInformeVentas(tipo: TipoVenta, desde: Date, hasta: Date): List<Venta>
+ventasPorDia(fecha: Date): List<Venta>
+ventasPorSemana(semana: int): List<Venta>
+ventasPorMes(mes: int): List<Venta>
+getTotalSubtotal(ventas: List<Venta>): double
+getTotalImpuestos(ventas: List<Venta>): double
+getTotalPropinas(ventas: List<Venta>): double
- mismoDia(a: Date, b: Date): boolean
Cafeteria
<<abstract>>
Alimento
-idProducto: String
-nombre: String
-precio: double
+getPrecio(): double
+getNombre(): String
+getIdProducto(): String
+puedeSerVendidoEnMesa(mesa: Mesa): boolean
Bebida
-Alcoholica: boolean
-Caliente: boolean
+isAlcoholica(): boolean
+isCaliente(): boolean
+puedeServirseEnMesa(mesa: Mesa): boolean
Pasteleria
-alergenos: List<String>
+getAlergenos(): List<String>
+contieneAlergeno(): boolean
+puedeServirseEnMesa(mesa: Mesa): boolean
Transacciones
SolicitudCambioTurno
+ CAMBIO_GENERAL = "CAMBIO_GENERAL" : String
+ INTERCAMBIO = "INTERCAMBIO": String
+ PENDIENTE = "PENDIENTE": String
+ APROBADA = "APROBADA": String
+ RECHAZADA = "RECHAZADA": String
-idSolicitud: String
-solicitante: Empleado
-turnoAfectado: Turno
-empleadoDestino: Empleado
-tipo: TipoSolicitud
-estado: EstadoSolicitud
+aprobar(): void
+rechazar(): void
+getEstado(): EstadoSolicitud
+getIdSolicitud(): String
+ getSolicitante(): Empleado
+ getTurnoAfectado(): Turno
+ getEmpleadoDestino(): Empleado
Turno
-idTurno: String
-diaSemana: String
-empleado: Empleado
+getDiaSemana(): String
+ getIdTurno(): String
+getEmpleado(): Empleado
+ setEmpleado(empleado: Empleado): void
Mesa
-idMesa: String
-ocupada: boolean
-numeroPersonas: int
-numeroNinos: int
-numeroJovenes: int
-prestamosActivos: List<Prestamo>
-pedidoActual: Pedido
+ocuparA(numPersonas: int, numNinos: int, numJovenes: int): void
+liberar(): void
+agregarPrestamo(prestamo: Prestamo): void
+getPrestamosActivos(): List<Prestamo>
+tieneJuegoDeAccion(): boolean
+hayNinos(): boolean
+hayJovenes(): boolean
+isOcupada(): boolean
+getNumeroPersonas(tablero): int
+ getNumeroPersonas(): int
+ tieneBebidaCaliente(): boolean
+ setPedidoActual(Pedido pedido): void
+getIdMesa(): String
Pedido
-idPedido: String
-mesa: Mesa
-items: List<ItemVenta>
- propina = 0.10: double
+agregarItem(Alimento alimento, int cantidad, double precioUnitario, double descuento): void
+ getSubtotalConImpuestos(): void
+ setPropina(double propina): void
+ getPropinaMonto(): double
+ getImpuestoConsumo(): double
+ getTotalFinal(): double
+ getIdPedido(): String
+ getMesa(): Mesa
+ getItems(): List<ItemVenta>
+ getPropina(): double
ItemVenta
- alimento: Alimento
- juego: JuegoDeMesa
- cantidad: int
- precioUnitario: double
- descuentoAplicado: double
+getSubtotal(): double
+ getAlimento(): Alimento
+ getJuego(): JuegoDeMesa
+ getCantidad(): int
+ getPrecioUnitario(): double
+ getDescuentoAplicado(): double
+ esDeJuego(): boolean
Venta
- idVenta: String
- fecha: Date
- comprador: Usuario
- items: List<ItemVenta>
- subtotal: double
- impuesto: double
- propina: double
- total: double
- TipoVenta {JUEGO, CAFETERA}
+agregarItem(item: ItemVenta): void
+calcularPuntosFidelidad(): double
+getSubTotal(): double
+getTipo(): TipoVenta
+ getIdVenta(): String
+ getComprador(): Usuario
+ getItems(): List<ItemVenta>
+ getFecha(): Date
+ getImpuesto(): double
+ getTotal(): double
+ getPropina(): doble
Prestamo
-idPrestamo: String
-copia: CopiaJuego
-usuario: Usuario
-mesa: Mesa
-fechaPrestamo: Date
-fechaDevolucion: Date
-activo: boolean
+devolver(): void
+isActivo(): boolean
+getJuego(): JuegoDeJuego
+ getUsuario(): Usuario
+ getMesa(): Mesa
+ getFechaPrestamo(): Date
+getFechaDevolucion(): Date
+ getIdPrestamo(): String
getTipo()
Inscripcion
- usuario: Usuario
-numParticipantes: int
- cuposFanaticosusados: int
- cuposRegularesUsados: int
- puedeRecibirPremioMetalico: boolean
- montoPagado: double
+ getUsuario(): Usuario
+ getNumParticipantes(): int
+getCuposFanaticosUsados(): int
+ getCuposRegularesUsados(): int
+ puedeRecibirPremioMetalico(): boolean
+getMontoPagado(): double
torneos
- idTorneo: String
- nombre: String
- juego: JuegoDeMesa
-diaSemana: String
- cupoMaximo: int
- inscripciones: List<Inscripcion>
+getIdTorneo() : String
+getNombre(): String
+ getJuego(): JuegoDeMesa
+ getDiaSemana(): String
+ getCupoMaximo(): int
+ getInscripciones(): List<Inscripcion>
+ cuposOcupados(): int
+ cuposDisponibles(): int
+ getCuposReservadosFanaticos(): int
+ cuposFanaticosOcupados(): int
+ cuposFanaticosDisponibles(): int
+ cuposRegularesOcupados(): int
+ getCuposRegularesTotales(): int
+ cuposRegularesDisponibles(): int
+usuarioEsFanatico(Usuario usuario): boolean
+ buscarInscripcion(Usuario usuario): Inscripcion
+agregarInscripcion(Inscripcion inscripcion): void
+ eliminarInscripcion(Usuario usuario): boolean
+esAmistoso(): boolean
+getTipo(): String
torneoCompetitivo
- tarifaEntrada: double
+ getTarifaEntrada(): double
+calcularPremio(): double
+ganadorPuedeRecibirPremioMetalico(Inscripcion inscripcionGanadora): boolean
+ esAmistoso(): boolean
+ getTipo(): String
torneoAmistoso
- bonoPremio: double
+ getBonoPremio(): double
+ esAmistoso(): boolean
+ getTipo(): String
Sistema
SistemaCafe
-usuarios: List<Usuario>
-mesas: List<Mesa>
-inventarioPrestamo: InventarioPrestamo
-inventarioVenta: InventarioVenta
-turnos: List<Turno>
-ventas: List<Venta>
-sugerencias: List<Sugerencia>
-servicioPrestamo: ServicioPrestamo
-servicioVentas: ServicioVentas
-servicioInventario: ServicioInventario
-servicioTurnos: ServicioTurnos
-servicioCafeteria: ServicioCafeteria
-servicioReportes: ServicioReportes
-servicioTorneo: ServicioTorneo
- capacidadMaxima: int
+loginDogin: String, password: String): Usuario
+prestarJuego(usuario: Usuario, juego: JuegoDeMesa, mesa: Mesa): Prestamo
+devolverJuego(prestamo: Prestamo): void
+crearVenta(comprador: Usuario, items: List<ItemVenta>): Venta
+realizarVentaCafeteria(comprador: Usuario, mesa: Mesa, items: List<ItemVenta>): Venta
+solicitarCambioTurno(solicitud: SolicitudCambioTurno): void
+aprobarCambio(solicitud: SolicitudCambioTurno): void
+agregarAlimento(alimento: Alimento): void
+aprobarSugerencia(sugerencia: Sugerencia): void
+generarInforme(tipo: TipoVenta, desde: Date, hasta: Date): List<Venta>
+ crearMesa(capacidad: int, numNinos: int, numJovenes: int, cliente: Cliente): Mesa
+ agregarUsuario(usuario: Usuario): void
+agregarMesa(mesa: Mesa): void
+liberarMesa(cliente: Cliente): void
+agregarSugerencia(sugerencia: Sugerencia): void
+ rechazarCambio(solicitud: SolicitudCambioTurno): void
+crearTorneoAmistoso(String idTorneo, String nombre, JuegoDeMesa juego, String diaSemana, int cupoMaximo, double bonoPremio): torneos
+crearTorneoCompetitivo(String idTorneo, String nombre, JuegoDeMesa juego, String diaSemana, int cupoMaximo, double tarifaEntrada): torneos
+inscribirseATorneo(torneos torneo, Usuario usuario, int numParticipantes): boolean
+ desinscribirseATorneo(torneos torneo, Usuario usuario): boolean
+registrarGanadorTorneo(torneos torneo, Usuario ganador): Inscripcion
+getTorneos(): List<torneos>
+ getServicioTorneo(): ServicioTorneo
+ getUsuarios(): List<Usuario>
+ getMesas(): List<Mesa>
+ getVentas(): List<Venta>
+ getSugerencias(): List<Sugerencia>
+ getServicioInventario(): ServicioInventario
+ getServicioPrestamo(): ServicioTurnos
+ getServicioTurnos(): ServicioTurnos
+ getServicioReportes(): ServicioReportes
+ getServicioCafeteria(): ServicioCafeteria
Persistencias
PersistenciaUsuarios
- SEP: String (static)
- SEP_LISTA: String (static)
- ARCHIVO: String (static)
- rutaDatos: String
+ guardar(usuarios: List<Usuario>): void
+ cargar():usuarios: List<Usuario>
- serializarCliente(c: Cliente): String
- serializarMesero(m: Mesero): String
- serializarCocinero(c: Cocinero): String
- serializarAdmin(a: Administrador): String
- serializarFavoritos(favs: List<JuegoDeMesa>): String
- cargarCliente(tp: String[], juegos: List<JuegoDeMesa>): Cliente
- cargarMesero(p: String[], juegos: List<JuegoDeMesa>): Mesero
- cargarCocinero(p: String[], juegos: List<JuegoDeMesa>): Cocinero
- cargarAdmin(p: String[], juegos: List<JuegoDeMesa>): Administrador
- cargarFavoritos(u: Usuario, campo: String, juegos: List<JuegoDeMesa>): void
- buscarJuego(juegos: List<JuegoDeMesa>, id: String): JuegoDeMesa
PersistenciaCafe
- SEP: String (static)
- ARCH_CAFE: String (static)
- ARCH_MESAS: String (static)
- ARCH_SUGERENCIAS: String (static)
- rutaDatos: String
- persJuegos: PersistenciaJuegos
- persCopias: PersistenciaCopias
- persUsuarios: PersistenciaUsuarios
- persMenu: PersistenciaMenu
- persTurnos: PersistenciaTurnos
- persPrestamos: PersistenciaPrestamos
- persVentas: PersistenciaVentas
+ leerCapacidadMaxima(rutaDatos: String): int (static)
+ guardar(sistema: SistemaCafe): void
+ cargar(sistema: SistemaCafe): boolean
- guardarConfigCafe(sistema: SistemaCafe): void
- guardarMesas(mesas: List<Mesa>): void
- cargarMesas(): List<Mesa>
- obtenerTodosLosJuegos(sistema: SistemaCafe): List<JuegoDeMesa>
- guardarSugerencias(sugerencias: List<Sugerencia>): void
- cargarSugerencias(usuarios: List<Usuario>): List<Sugerencia>
PersistenciaJuegos
- SEP: String (static)
- ARCHIVO: String (static)
- rutaDatos: String
+ guardar(juegos: List<JuegoDeMesa>): void
+ cargar(): List<JuegoDeMesa>
PersistenciaMenu
- SEP: String (static)
- SEP_LISTA: String (static)
- ARCHIVO: String (static)
- rutaDatos: String
+ guardar(menu: List<Alimento>): void
+ cargar(): List<Alimento>
Juegos
JuegoDeMesa
-idJuego: String
-nombre: String
-anioPublicacion: int
-empresaMatriz: String
-minJugadores: int
-maxJugadores: int
-edadMinima: int
-categoria: String
-dificil: boolean
+ getNombre(): String
+ getCategoria(): String
+ esAptoParaEdad(): boolean
+ esAptoParaNPersonas(int numPersonas): boolean
+ esAptoParaEdad(boolean hayNinos, boolean hayJovenes): boolean
+ esCompatibleConBebidaCaliente(): boolean
+ getIdJuego(): String
CopiaJuego
-idCopia: String
-juego: JuegoDeMesa
-estadoCopia: String
-disponible: boolean
+marcarDisponible(): void
+marcarNoDisponible(): void
+actualizarEstado(estado: EstadoCopia): void
+isDisponible(): boolean
+getJuego(): JuegoDeMesa
+ getIdCopia(): String
+ getEstadoCopia(): String
Inventario
InventarioVenta
-stock: Map<JuegoDeMesa, Integer>
-precios: Map<JuegoDeMesa, Double>
+vender(juego: JuegoDeMesa, cantidad: int): void
+reabastecer(juego: JuegoDeMesa, cantidad: int, precio: double): void
+getPrecio(juego: JuegoDeMesa): double
+hayStock(juego: JuegoDeMesa): boolean
+ getCantidad(juego: JuegoDeMesa): int
+ getJuegos(): Set<JuegoDeMesa>
InventarioPrestamo
-copias: List<CopiaJuego>
+agregarCopia(copia: CopiaJuego): void
+retirarCopia(copia: CopiaJuego): void
+getCopiaDisponible(juego: JuegoDeMesa): CopiaJuego
+getCopiasPorJuego(juego: JuegoDeMesa): List<CopiaJuego>
+ getCopias(): List<CopiaJuego>
PersistenciaTurnos
- SEP: String (static)
- NULO: String (static)
- ARCH_TURNOS: String (static)
- ARCH_SOLICITUDES: String (static)
- rutaDatos: String
+ guardarTurnos(turnos: List<Turno>): void
+ cargarTurnos(usuarios: List<Usuario>): List<Turno>
+ guardarSolicitudes(solicitudes: List<SolicitudCambioTurno>): void
+ cargarSolicitudes(usuarios: List<Usuario>, turnos: List<Turno>): List<SolicitudCambioTurno>
- buscarUsuario(usuarios: List<Usuario>, id: String): Usuario
- buscarTurno(turnos: List<Turno>, idTurno: String): Turno
PersistenciaVentas
- SEP: String (static)
- SEP_ITEM: String (static)
- SEP_CAMPO: String (static)
- FECHA_FMT: String (static)
- ARCHIVO: String (static)
- rutaDatos: String
+ guardar(ventas: List<Venta>): void
+ cargar(usuarios: List<Usuario>, juegos: List<JuegoDeMesa>, menu: List<Alimento>): List<Venta>
- serializarItems(items: List<ItemVenta>): String
- cargarItems(itemsStr: String, juegos: List<JuegoDeMesa>, menu: List<Alimento>): List<ItemVenta>
- parseFecha(sdf: SimpleDateFormat, s: String): Date
- buscarUsuario(usuarios: List<Usuario>, id: String): Usuario
- buscarJuego(juegos: List<JuegoDeMesa>, id: String): JuegoDeMesa
- buscarAliment(menu: List<Alimento>, id: String): Alimento
PersistenciaPrestamos
- SEP: String (static)
- NULO: String (static)
- FECHA_FMT: String (static)
- ARCHIVO: String (static)
- rutaDatos: String
+ guardar(prestamos: List<Prestamo>): void
+ cargar(copias: List<CopiaJuego>, usuarios: List<Usuario>, mesas: List<Mesa>): List<Prestamo>
- parseFecha(sdf: SimpleDateFormat, s: String): Date
- buscarCopia(copias: List<CopiaJuego>, id: String): CopiaJuego
- buscarUsuario(usuarios: List<Usuario>, id: String): Usuario
- buscarMesa(mesas: List<Mesa>, id: String): Mesa
PersistenciaCopias
- SEP: String (static)
- ARCHIVO_PRESTAMO: String (static)
- ARCHIVO_VENTA: String (static)
- rutaDatos: String
+ guardarCopiasPrestamo(inv: InventarioPrestamo): void
+ cargarCopiasPrestamo(inv: InventarioPrestamo, juegos: List<JuegoDeMesa>): void
+ guardarInventarioVenta(inv: InventarioVenta): void
+ cargarInventarioVenta(inv: InventarioVenta, juegos: List<JuegoDeMesa>): void
- buscarJuego(juegos: List<JuegoDeMesa>, idJuego: String): JuegoDeMesa
1
*
0..1
0..*